MySQL

Basic → Advanced Notes

Syntax-first reference for SQL, database design, joins, aggregation, subqueries, CTEs, window functions, indexes, transactions, locking, views, procedures, triggers, JSON, optimization and interview preparation.

Level	Topics
Beginner	Database basics, DDL, DML, SELECT, filtering, sorting, NULL, functions
Intermediate	GROUP BY, HAVING, joins, subqueries, CTEs, set operations, constraints
Advanced	Indexes, EXPLAIN, transactions, isolation, locking, views, procedures, triggers
Modern MySQL	Window functions, CTEs, recursive CTEs, JSON, generated columns
Interview	Common traps, query patterns, performance checklist

Examples use MySQL 8.x syntax. Always verify version-specific behavior in your environment.

1. Database & SQL Fundamentals

Database = organized collection of data. RDBMS stores data in related tables. MySQL is an RDBMS.

SQL is the language used to define, query and modify relational data.

Mental model

Typical backend flow: Application → SQL driver/ORM → MySQL → storage engine. In MySQL, InnoDB is the common transactional storage engine.

2. Create Database, Table & Data Types

Create and select a database:

```
CREATE DATABASE shop;
USE shop;
```

Common data types:

Type	Use	Example
INT / BIGINT	Whole numbers / IDs	id INT
DECIMAL(p,s)	Exact money/precision	price DECIMAL(10,2)
VARCHAR(n)	Variable-length text	name VARCHAR(100)
TEXT	Large text	description TEXT
DATE	Date only	dob DATE
DATETIME	Date + time	created_at DATETIME
TIMESTAMP	Timestamp values	updated_at TIMESTAMP
BOOLEAN	True/false style	active BOOLEAN
JSON	JSON documents	metadata JSON

Create a practical table:

```
CREATE TABLE users (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(255) NOT NULL UNIQUE,
    age INT,
    salary DECIMAL(12,2),
    active BOOLEAN DEFAULT TRUE,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

Tip: Use DECIMAL for exact financial values instead of FLOAT/DOUBLE.

3. Constraints & Keys

Constraints protect data integrity.

Constraint	Meaning
PRIMARY KEY	Uniquely identifies a row; cannot be NULL
FOREIGN KEY	Links a child column to a parent key
UNIQUE	Prevents duplicate values
NOT NULL	Value must be supplied
DEFAULT	Supplies a value when omitted
CHECK	Enforces a condition

Primary key + foreign key relationship:

```
CREATE TABLE departments (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) UNIQUE NOT NULL
);

CREATE TABLE employees (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    dept_id INT,
    salary DECIMAL(10,2) CHECK (salary >= 0),
    FOREIGN KEY (dept_id) REFERENCES departments(id)
);
```

Candidate key: a column/set of columns that can uniquely identify a row. Composite key: key made from multiple columns.

4. INSERT, UPDATE, DELETE

INSERT

```
INSERT INTO users (name, email, age)
VALUES ('Akash', 'akash@example.com', 24);

INSERT INTO users (name, email, age)
VALUES
('Aman', 'aman@example.com', 23),
('Riya', 'riya@example.com', 25);
```

UPDATE

```
UPDATE users
SET age = 25
WHERE id = 1;
```

DELETE

```
DELETE FROM users
WHERE id = 1;
```

Danger: UPDATE/DELETE without WHERE affects every matching row—possibly the whole table.

Useful pattern:

```
UPDATE users
SET salary = salary * 1.10
WHERE dept_id = 2
  AND active = TRUE;
```

5. SELECT — Core Query Syntax

```
SELECT column1, column2
FROM table_name
WHERE condition
ORDER BY column1 ASC
LIMIT 10;
```

Example:

```
SELECT id, name, salary
FROM employees
WHERE salary >= 50000
ORDER BY salary DESC
LIMIT 5;
```

Aliases:

```
SELECT e.name AS employee_name,
       e.salary AS monthly_salary
FROM employees AS e;
```

DISTINCT removes duplicate result values:

```
SELECT DISTINCT dept_id
FROM employees;
```

Logical processing order to remember: FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT. (Actual optimizer execution can differ.)

6. WHERE, Operators & NULL

Operators

Type	Examples
Comparison	=, <>, !=, >, <, >=, <=
Logical	AND, OR, NOT
Range	BETWEEN ... AND ...
Set	IN (...), NOT IN (...)
Pattern	LIKE, NOT LIKE
NULL	IS NULL, IS NOT NULL

```
SELECT *
FROM employees
WHERE salary BETWEEN 40000 AND 80000
  AND dept_id IN (1, 2, 3)
  AND name LIKE 'A%';
```

NULL is not 0 and not an empty string. Use IS NULL / IS NOT NULL, not = NULL.

```
-- Wrong
WHERE manager_id = NULL

-- Correct
WHERE manager_id IS NULL
```

Three-valued logic: comparisons involving NULL generally produce UNKNOWN, which does not pass WHERE.

7. Built-in Functions

String functions

```
SELECT UPPER(name), LOWER(email),
       LENGTH(name), CONCAT(name, ' - ', email)
FROM users;
```

Numeric functions

```
SELECT ROUND(123.456, 2),
       CEIL(12.2),
       FLOOR(12.9),
       ABS(-10);
```

Date/time functions

```
SELECT NOW(), CURDATE(), CURTIME();

SELECT DATEDIFF(CURDATE(), DATE(created_at))
FROM users;

SELECT DATE_FORMAT(created_at, '%Y-%m-%d')
FROM users;
```

NULL handling

```
SELECT COALESCE(phone, 'Not provided')
FROM users;
```

COALESCE(a,b,c) returns the first non-NULL expression.

8. CASE Expressions

CASE is SQL's conditional expression.

```
SELECT name, salary,
       CASE
           WHEN salary >= 100000 THEN 'HIGH'
           WHEN salary >= 60000  THEN 'MEDIUM'
           ELSE 'LOW'
       END AS salary_band
FROM employees;
```

Conditional aggregation:

```
SELECT
    COUNT(*) AS total,
    SUM(CASE WHEN active = TRUE THEN 1 ELSE 0 END) AS active_users,
    SUM(CASE WHEN active = FALSE THEN 1 ELSE 0 END) AS inactive_users
FROM users;
```

CASE is useful for reporting, bucketing, conditional counts and custom labels.

9. Aggregate Functions, GROUP BY & HAVING

Function	Purpose
COUNT(*)	Count rows
COUNT(column)	Count non-NULL values
SUM()	Total
AVG()	Average
MIN()/MAX()	Minimum/maximum

```
SELECT dept_id,
       COUNT(*) AS employee_count,
       AVG(salary) AS avg_salary,
       MAX(salary) AS max_salary
FROM employees
GROUP BY dept_id;
```

HAVING filters groups after aggregation:

```
SELECT dept_id, AVG(salary) AS avg_salary
FROM employees
GROUP BY dept_id
HAVING AVG(salary) > 70000;
```

WHERE filters rows before grouping. HAVING filters groups after grouping.

10. JOINs — The Most Important Topic

Assume: employees(dept_id) → departments(id).

JOIN	Returns
INNER JOIN	Only matching rows
LEFT JOIN	All left rows + matches
RIGHT JOIN	All right rows + matches
CROSS JOIN	Cartesian product
SELF JOIN	Table joined to itself

INNER JOIN

```
SELECT e.name, d.name AS department
FROM employees e
INNER JOIN departments d
    ON e.dept_id = d.id;
```

LEFT JOIN — keep employees even when department is missing:

```
SELECT e.name, d.name AS department
FROM employees e
LEFT JOIN departments d
    ON e.dept_id = d.id;
```

Common trap: putting a right-table condition in WHERE can turn a LEFT JOIN into effectively an INNER JOIN.

11. SELF JOIN & Many-to-Many

SELF JOIN example: employees have managers stored in the same table.

```
SELECT e.name AS employee,
       m.name AS manager
FROM employees e
LEFT JOIN employees m
    ON e.manager_id = m.id;
```

Many-to-many relationships use a junction table.

```
CREATE TABLE students (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL
);

CREATE TABLE courses (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL
);

CREATE TABLE student_courses (
    student_id INT,
    course_id INT,
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES students(id),
    FOREIGN KEY (course_id) REFERENCES courses(id)
);
```

Query students with their courses:

```
SELECT s.name AS student, c.name AS course
FROM student_courses sc
JOIN students s ON s.id = sc.student_id
JOIN courses c ON c.id = sc.course_id;
```

12. Subqueries

A subquery is a query nested inside another query.

Scalar subquery:

```
SELECT name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

IN subquery:

```
SELECT name
FROM employees
WHERE dept_id IN (
    SELECT id
    FROM departments
    WHERE name IN ('Engineering', 'Finance')
);
```

EXISTS — checks whether at least one related row exists:

```
SELECT d.name
FROM departments d
WHERE EXISTS (
    SELECT 1
    FROM employees e
    WHERE e.dept_id = d.id
);
```

EXISTS is often a good fit when you only care whether a related row exists, not its values.

13. CTEs — WITH

A CTE (Common Table Expression) names a temporary result for one statement.

```
WITH high_paid AS (
    SELECT id, name, salary, dept_id
    FROM employees
    WHERE salary >= 80000
)
SELECT *
FROM high_paid
ORDER BY salary DESC;
```

Multiple CTEs:

```
WITH dept_stats AS (
    SELECT dept_id, AVG(salary) AS avg_salary
    FROM employees
    GROUP BY dept_id
),
big_depts AS (
    SELECT dept_id
    FROM employees
    GROUP BY dept_id
    HAVING COUNT(*) >= 5
)
SELECT d.dept_id, d.avg_salary
FROM dept_stats d
JOIN big_depts b ON b.dept_id = d.dept_id;
```

CTEs improve readability and can make complex SQL easier to debug. They are not automatically faster than equivalent queries.

14. Recursive CTEs

Recursive CTEs are useful for hierarchies such as employee → manager or category trees.

```
WITH RECURSIVE org AS (
    -- Anchor
    SELECT id, name, manager_id, 0 AS level
    FROM employees
    WHERE manager_id IS NULL

    UNION ALL

    -- Recursive part
    SELECT e.id, e.name, e.manager_id, o.level + 1
    FROM employees e
    JOIN org o
      ON e.manager_id = o.id
)
SELECT *
FROM org
ORDER BY level, id;
```

Structure: anchor query → UNION ALL → recursive query. Always make sure the recursion can terminate.

15. Set Operations

MySQL supports set-style combination through UNION / UNION ALL.

```
SELECT email FROM customers
UNION
SELECT email FROM suppliers;

SELECT email FROM customers
UNION ALL
SELECT email FROM suppliers;
```

Operator	Behavior
UNION	Combines results and removes duplicates
UNION ALL	Combines results and keeps duplicates

Queries combined by UNION must have compatible column counts and compatible types/semantics in corresponding positions.

16. Views

A view is a stored query that can be queried like a table.

```
CREATE VIEW employee_summary AS
SELECT e.id, e.name, d.name AS department, e.salary
FROM employees e
LEFT JOIN departments d ON d.id = e.dept_id;

SELECT *
FROM employee_summary
WHERE salary > 70000;
```

Modify/drop:

```
CREATE OR REPLACE VIEW employee_summary AS
SELECT id, name, salary
FROM employees;

DROP VIEW employee_summary;
```

Views are useful for reusable reporting logic and controlled exposure of columns. A normal view does not mean the result is physically stored like a table.

17. Indexes

An index is a data structure that can speed up lookups, filtering, joins and ordering—but indexes also consume storage and add write/update overhead.

```
CREATE INDEX idx_users_email
ON users(email);

CREATE INDEX idx_emp_dept_salary
ON employees(dept_id, salary);

CREATE UNIQUE INDEX idx_users_email_unique
ON users(email);
```

Composite index:

```
CREATE INDEX idx_orders_customer_date
ON orders(customer_id, created_at);
```

Rule of thumb: column order matters. An index on (A, B) is naturally useful for access patterns beginning with A; it is not equivalent to separate indexes on A and B.

Avoid blindly indexing every column. Evaluate real query patterns and write cost.

18. EXPLAIN & Query Performance

```
EXPLAIN
SELECT e.name, d.name
FROM employees e
JOIN departments d ON d.id = e.dept_id
WHERE e.salary > 80000;
```

Important EXPLAIN ideas:

Field/concept	What to inspect
type	Access strategy; avoid unexpectedly expensive scans when selective access is possible
possible_keys	Indexes optimizer could consider
key	Index actually selected
rows	Estimated rows examined
Extra	Additional execution details such as filtering, temporary work or sorting

Use EXPLAIN ANALYZE when available to inspect actual execution behavior, not only estimates.

```
EXPLAIN ANALYZE
SELECT *
FROM orders
WHERE customer_id = 42
ORDER BY created_at DESC
LIMIT 20;
```

19. Transactions & ACID

Property	Meaning
Atomicity	All changes in a transaction succeed or are rolled back
Consistency	Constraints/rules remain satisfied
Isolation	Concurrent transactions are controlled from interfering incorrectly
Durability	Committed changes survive failures, subject to system guarantees

```
START TRANSACTION;

UPDATE accounts
SET balance = balance - 1000
WHERE id = 1;

UPDATE accounts
SET balance = balance + 1000
WHERE id = 2;

COMMIT;
```

Rollback:

```
START TRANSACTION;

UPDATE accounts
SET balance = balance - 1000
WHERE id = 1;

ROLLBACK;
```

Savepoints:

```
START TRANSACTION;

UPDATE accounts SET balance = balance - 100 WHERE id = 1;
SAVEPOINT after_debit;

UPDATE accounts SET balance = balance + 100 WHERE id = 2;

ROLLBACK TO SAVEPOINT after_debit;
COMMIT;
```

20. Isolation Levels & Concurrency

Common phenomena:

Phenomenon	Meaning
Dirty read	Reading uncommitted data
Non-repeatable read	Same row gives different committed values within a transaction
Phantom	A repeated range query sees a changed set of rows

Standard isolation levels:

Level	General idea
READ UNCOMMITTED	Allows dirty reads
READ COMMITTED	Reads committed data; weaker than repeatable read
REPEATABLE READ	Provides stronger repeatability; common/default behavior in InnoDB
SERIALIZABLE	Strongest isolation; more concurrency restrictions

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

Exact locking/visibility behavior depends on engine and query shape. For InnoDB, understand MVCC, row locks and next-key/gap locking for interviews.

21. Locking — SELECT FOR UPDATE

Use row-locking patterns when a transaction must read a value and then safely update it.

```
START TRANSACTION;

SELECT id, balance
FROM accounts
WHERE id = 1
FOR UPDATE;

UPDATE accounts
SET balance = balance - 500
WHERE id = 1;

COMMIT;
```

Conceptual flow:

```
Transaction A                  Transaction B
     |                              |
SELECT ... FOR UPDATE               |
     |                        tries same row
row locked                          |
     |                         X waits
UPDATE                              |
     |                              |
COMMIT ---------------------------> lock released
                                 B continues
```

Deadlock: two transactions can wait for each other. InnoDB detects deadlocks and rolls back one transaction; application code should be prepared to retry appropriate operations.

22. Stored Procedures & Functions

Stored procedure:

```
DELIMITER //

CREATE PROCEDURE get_employee(IN emp_id INT)
BEGIN
    SELECT id, name, salary
    FROM employees
    WHERE id = emp_id;
END //

DELIMITER ;

CALL get_employee(10);
```

Stored function:

```
DELIMITER //

CREATE FUNCTION annual_salary(monthly DECIMAL(10,2))
RETURNS DECIMAL(12,2)
DETERMINISTIC
RETURN monthly * 12 //

DELIMITER ;

SELECT annual_salary(50000);
```

Use stored programs when database-side encapsulation is useful. In many application architectures, complex business logic is kept in application code instead.

23. Triggers

A trigger runs automatically when a specified table event occurs.

```
CREATE TABLE audit_log (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    user_id BIGINT,
    action VARCHAR(50),
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

DELIMITER //

CREATE TRIGGER after_user_insert
AFTER INSERT ON users
FOR EACH ROW
BEGIN
    INSERT INTO audit_log(user_id, action)
    VALUES (NEW.id, 'USER_CREATED');
END //

DELIMITER ;
```

NEW refers to the new row; OLD refers to the previous row where applicable. Triggers can make behavior less visible to application developers, so document them carefully.

24. JSON in MySQL

MySQL can store JSON documents in a JSON column.

```
CREATE TABLE products (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100),
    attributes JSON
);

INSERT INTO products(name, attributes)
VALUES (
    'Laptop',
    '{"brand":"ABC","ram":16,"tags":["work","dev"]}'
);
```

Read JSON values:

```
SELECT
    name,
    JSON_EXTRACT(attributes, '$.brand') AS brand,
    JSON_EXTRACT(attributes, '$.ram') AS ram
FROM products;
```

JSON path examples:

```
$.brand
$.ram
$.tags[0]
```

For frequently filtered relational attributes, a normal column (possibly indexed) can be more straightforward and performant than hiding everything inside JSON.

25. Normalization & Database Design

Normalization reduces redundancy and update anomalies.

Normal Form	Simplified idea
1NF	Atomic values; no repeating groups
2NF	1NF + non-key attributes depend on the whole key
3NF	2NF + non-key attributes do not depend transitively on the key

Bad design:

```
orders(
    id,
    customer_name,
    customer_phone,
    product1,
    product2,
    product3
)
```

More normalized design:

```
customers(id, name, phone)
orders(id, customer_id, created_at)
products(id, name, price)
order_items(order_id, product_id, quantity)
```

Normalization is not a religion: controlled denormalization can be useful for performance or reporting when the trade-off is understood.

26. Pagination Patterns

Offset pagination:

```
SELECT id, name, created_at
FROM users
ORDER BY id
LIMIT 20 OFFSET 40;
```

Simple, but large offsets can require the database to walk past many rows.

Keyset/seek pagination:

```
SELECT id, name, created_at
FROM users
WHERE id > 1000
ORDER BY id
LIMIT 20;
```

For stable pagination, use a deterministic ordering. If ordering by multiple columns, the cursor condition should account for the full ordering tuple.

27. Top-N & Window Functions

Window functions calculate across related rows without collapsing them like GROUP BY.

```
SELECT
    name,
    dept_id,
    salary,
    ROW_NUMBER() OVER (
        PARTITION BY dept_id
        ORDER BY salary DESC
    ) AS rn
FROM employees;
```

Top 3 employees per department:

```
WITH ranked AS (
    SELECT
        id, name, dept_id, salary,
        ROW_NUMBER() OVER (
            PARTITION BY dept_id
            ORDER BY salary DESC
        ) AS rn
    FROM employees
)
SELECT *
FROM ranked
WHERE rn <= 3;
```

Useful window functions: ROW_NUMBER(), RANK(), DENSE_RANK(), LAG(), LEAD(), SUM() OVER, AVG() OVER.

28. Running Total, LAG & LEAD

Running total:

```
SELECT
    order_date,
    amount,
    SUM(amount) OVER (
        ORDER BY order_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS running_total
FROM daily_sales;
```

Compare with previous row:

```
SELECT
    order_date,
    amount,
    LAG(amount) OVER (ORDER BY order_date) AS previous_amount,
    amount - LAG(amount) OVER (ORDER BY order_date) AS difference
FROM daily_sales;
```

GROUP BY reduces rows into groups. Window functions keep the individual rows and add calculations beside them.

29. ALTER TABLE & Schema Changes

```
ALTER TABLE users
ADD COLUMN phone VARCHAR(20);

ALTER TABLE users
MODIFY COLUMN phone VARCHAR(30);

ALTER TABLE users
RENAME COLUMN phone TO mobile;

ALTER TABLE users
DROP COLUMN mobile;
```

Add an index or constraint:

```
ALTER TABLE users
ADD UNIQUE KEY uq_users_email (email);

ALTER TABLE employees
ADD CONSTRAINT fk_emp_dept
FOREIGN KEY (dept_id) REFERENCES departments(id);
```

Schema changes on large production tables can lock or rebuild data depending on the operation/version/algorithm. Plan migrations carefully.

30. Transactions vs DDL — Practical Caution

- Test the exact ALTER/CREATE/DROP operation on the same MySQL version and table shape.
- Estimate table size and migration duration.
- Check locking and availability impact.
- Have a rollback/restore strategy.
- Use migration tooling in application projects rather than manual ad-hoc production changes.

Not every SQL statement behaves like a simple row update. Some schema operations can cause implicit commits or have special transactional behavior.

For production migrations:

Typical Java/Spring Boot stack: Spring Boot → JPA/JDBC → MySQL, with migrations commonly managed by tools such as Flyway or Liquibase.

31. Security & Permissions

- Use least privilege.
- Do not use the root account from application code.
- Keep credentials outside source code.
- Use parameterized queries/prepared statements.
- Restrict network exposure and secure connections appropriately.

Users and privileges:

```
CREATE USER 'app_user'@'localhost'
IDENTIFIED BY 'strong-password';

GRANT SELECT, INSERT, UPDATE, DELETE
ON shop.*
TO 'app_user'@'localhost';

SHOW GRANTS FOR 'app_user'@'localhost';
```

Principles:

SQL injection prevention:

```
-- Prefer parameterized SQL:
SELECT id, name
FROM users
WHERE email = ?;
```

32. SQL Injection — What NOT to Do

Never concatenate untrusted input into SQL.

```
-- Unsafe idea:
sql = "SELECT * FROM users WHERE email = '" + email + "'";
```

Parameterized query concept:

```
-- SQL:
SELECT * FROM users WHERE email = ?;

-- Parameter:
email = "akash@example.com"
```

ORMs such as JPA/Hibernate can still be used unsafely if developers construct dynamic query fragments carelessly. Parameterization remains important.

33. Important Query Patterns

Second highest salary:

```
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Employees earning above department average:

```
SELECT e.*
FROM employees e
JOIN (
    SELECT dept_id, AVG(salary) AS avg_salary
    FROM employees
    GROUP BY dept_id
) d ON d.dept_id = e.dept_id
WHERE e.salary > d.avg_salary;
```

Find duplicates:

```
SELECT email, COUNT(*) AS cnt
FROM users
GROUP BY email
HAVING COUNT(*) > 1;
```

Rows with no matching child:

```
SELECT d.*
FROM departments d
LEFT JOIN employees e ON e.dept_id = d.id
WHERE e.id IS NULL;
```

34. More Interview Query Patterns

Nth highest salary using DENSE_RANK:

```
WITH ranked AS (
    SELECT salary,
           DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employees
)
SELECT DISTINCT salary
FROM ranked
WHERE rnk = 3;
```

Customers with at least 3 orders:

```
SELECT customer_id, COUNT(*) AS order_count
FROM orders
GROUP BY customer_id
HAVING COUNT(*) >= 3;
```

Latest order per customer:

```
WITH ranked AS (
    SELECT o.*,
           ROW_NUMBER() OVER (
               PARTITION BY customer_id
               ORDER BY created_at DESC, id DESC
           ) AS rn
    FROM orders o
)
SELECT *
FROM ranked
WHERE rn = 1;
```

35. MySQL Performance Checklist

When a query is slow, work systematically:

- 1. Confirm the exact slow query and parameters.
- 2. Run EXPLAIN / EXPLAIN ANALYZE.
- 3. Check rows examined vs rows returned.
- 4. Check indexes for WHERE/JOIN/ORDER BY patterns.
- 5. Avoid SELECT * when unnecessary.
- 6. Filter early when it reduces work.
- 7. Review joins and accidental row multiplication.
- 8. Check sorting/grouping and temporary work.
- 9. Check table statistics and data distribution.
- 10. Consider schema/query redesign before blindly adding indexes.

Example:

```
-- Query
SELECT id, customer_id, created_at
FROM orders
WHERE customer_id = 42
ORDER BY created_at DESC
LIMIT 20;

-- Potential composite index
CREATE INDEX idx_orders_customer_created
ON orders(customer_id, created_at);
```

36. JOIN Cardinality & Duplicate Rows

A common production bug is getting more rows than expected after a JOIN.

```
customers: 1 row per customer
orders:    many rows per customer

customers JOIN orders
=> one customer can appear many times
```

If you need one row per customer, aggregate or rank deliberately:

```
SELECT c.id, c.name, COUNT(o.id) AS order_count
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
GROUP BY c.id, c.name;
```

Do not add DISTINCT just to hide duplicate rows until you understand why the join multiplied them.

37. NULL & COUNT — Interview Trap

```
CREATE TABLE t (
    id INT,
    value INT
);

-- Suppose values are: 10, NULL, 20

SELECT COUNT(*) FROM t;       -- 3
SELECT COUNT(value) FROM t;  -- 2
SELECT SUM(value) FROM t;    -- 30
SELECT AVG(value) FROM t;    -- 15
```

COUNT(*) counts rows. COUNT(column) counts non-NULL values in that column. Most aggregate functions ignore NULL values.

Use COALESCE when you need a default:

```
SELECT COALESCE(SUM(value), 0)
FROM t
WHERE id > 100;
```

38. DELETE vs TRUNCATE vs DROP

Command	Effect
DELETE	Removes selected rows; supports WHERE
TRUNCATE TABLE	Removes all rows efficiently; no WHERE
DROP TABLE	Removes the table definition and its data

```
DELETE FROM users WHERE id = 10;

TRUNCATE TABLE users;

DROP TABLE users;
```

TRUNCATE and DROP are destructive. Understand foreign keys, auto-increment behavior, permissions and transactional/DDL semantics before using them in production.

39. MySQL Data Modeling for Backend Systems

Typical e-commerce schema:

Entity	Typical important columns
users	id, email, password_hash, created_at
orders	id, user_id, status, total, created_at
order_items	order_id, product_id, quantity, unit_price
products	id, name, price, stock
payments	id, order_id, status, provider_ref

For money, store exact values with DECIMAL or use integer minor units (e.g., paise/cents) depending on the system design.

40. Final One-Shot Revision Sheet

Topic	Remember
SELECT	SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY ... LIMIT
JOIN	INNER = matches; LEFT = preserve left side
WHERE	Filters rows
HAVING	Filters groups
GROUP BY	Collapses rows into groups
CTE	WITH name AS (...)
Window	Calculates across rows without collapsing them
Index	Speeds reads, costs storage/write overhead
EXPLAIN	Inspect optimizer plan
Transaction	START TRANSACTION → COMMIT/ROLLBACK
Lock	FOR UPDATE for appropriate transactional row locking
NULL	Use IS NULL; COUNT(col) ignores NULL
PK	Unique row identity
FK	Relationship + referential integrity
UNIQUE	No duplicate values
DELETE	Rows; WHERE allowed
TRUNCATE	All rows
DROP	Object + data
UNION	Deduplicates
UNION ALL	Keeps duplicates

Backend interview priority: SELECT → WHERE → JOIN → GROUP BY/HAVING → subqueries → indexes/EXPLAIN → transactions/isolation → window functions → CTEs → schema design.

Practice rule: Don't only read SQL. Write queries against a small schema, insert realistic data, then inspect the results and EXPLAIN plans.